

Building OntarioDriveTestMap

The project, the stack, and everything that broke on the way

Zaid Ahmad

ontariodrivetestmap.fyi

Project report · September 2026

Abstract

OntarioDriveTestMap is a free website that shows the routes used in Ontario's G2 and G road tests. The routes are not published anywhere official, so the site builds them from what test-takers leave behind in public: Reddit write-ups and dashcam videos on YouTube. It draws each route on an interactive map with turn-by-turn directions, stop signs, traffic lights and speed limits, and it marks which parts are confirmed by sources and which are inferred.

This report is the engineering story, not the statistics. It covers what the site does, the stack it runs on and how that stack drifted from the plan, and, at more length, the problems hit along the way. The project ran solo, from first planning on 28 August 2026 to a deployed, public site four weeks later. Most of what is worth telling happened between those two points. Three Reddit access routes failed. An OCR run reported "no text" because it had crashed on every frame. A route passed every automated check while missing an entire street. The same name-normalisation bug turned up five times in five files. A whole centre's worth of video turned out to be from Winchester, England. And a session cookie quietly logged every user out the moment the site moved to its real domain.

1 The project

1.1 The problem

Ontario's road tests run out of DriveTest centres, operated by Serco on behalf of the Ministry of Transportation. Examiners drive set routes, and those routes are not published. Plenty of websites claim to show them. A survey of the main ones during the project found that none had real route data. The best-known paid site says in its own disclaimer that its free sample route is not an actual test route. Another labels its map a schematic, not to scale. Free is table stakes in this space. Real data is the only thing that sets a site apart.

The data does exist. People write up their test on Reddit, sometimes turn by turn. Instructors and test-takers post dashcam recordings to YouTube. Any single account is partial, some copy others, and nothing official confirms any of them. The project's bet was that many weak accounts, handled honestly, add up to something useful, as long as the site always shows how sure it is.

1.2 What the site does today

- **Route map.** Each centre has its G and G2 routes, one route drawn at a time and the most-corroborated first. The line uses one notation everywhere: *Confirmed* (solid teal), *Inferred*

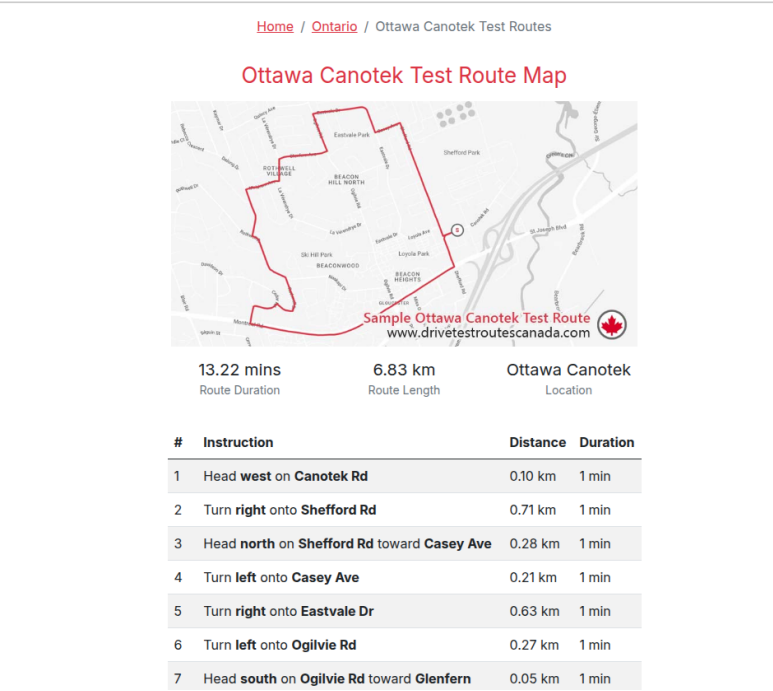


Figure 1: A competitor’s page for Ottawa Canotek. The map is captioned as a sample route. Its turn-by-turn table (number, instruction, distance, duration) later became the reference layout for this site’s own directions, though not its data.

(amber dash) and *Unrouted gap* (grey dots). Routes that were hand-verified from a real test video carry their own badge.

- **Turn-by-turn directions**, written in plain language and filled with real per-leg distances, stop signs, traffic lights and speed limits taken from OpenStreetMap. Numbered turn pins on the map link both ways to the direction list.
- **Route playback**. A dot travels the drawn route, with play, scrub and speed controls, and hops across unrouted gaps rather than inventing a road.
- **Mobile practice drive**. A full-screen GPS follower for use in a moving car. It shows GPS quality separately from route confidence, and location never leaves the phone.
- **Community**. Route submissions through a four-step wizard that checks each street against the real road graph as it is entered. A “Tips and corrections” forum per centre, a general Discussion board with nested replies, @mentions, image uploads and in-app notifications.
- **Everything else**. English and French, GPX export and print, SEO pages, real privacy/terms/cookie pages, self-service account deletion and a booking-check reminder. Guests can read every route; signing in is needed only to post.

1.3 Launch centres

Four centres were chosen on purpose. **Walkley** and **Canotek** in Ottawa have the most public material, so they prove the pipeline. **Smiths Falls** and **Winchester** are small rural centres that test what happens when data is thin. Walkley, Canotek and Smiths Falls shipped. Winchester was

skipped, for reasons in Section 4.5.

2 The stack

The constraints shaped everything. It was a solo project. The development machine was a laptop with 7 GB of RAM and integrated graphics, which rules out large local speech models, province-wide routing preprocessing and a self-hosted geocoder. The running budget was about \$10 a month, with no ads.

2.1 Planned against built

Table 1: The stack as designed on day one, and as it actually runs.

Layer	Planned	Built
Reddit data	Official API or public JSON	Threads saved by hand and parsed offline, after three access routes failed
Video data	YouTube Data API, yt-dlp, ffmpeg	Same, with downloads moved to a home connection (Section 4.1)
Speech	Captions, Whisper fallback	Captions tried and dropped; no Whisper
On-screen text	PaddleOCR, secondary path	PaddleOCR on a free Colab T4 GPU, now the primary path
Geocoding	Nominatim	None. A local intersection index built from OpenStreetMap with pyosmium, stored in SQLite
Routing	Self-hosted OSRM in Docker	Public OSRM server for the handful of routes needed
Clustering	HDBSCAN	HDBSCAN, partitioned by test class
Database	Postgres + PostGIS	Postgres on Railway, no PostGIS; plain lat/lon columns and JSONB geometry
API	FastAPI	FastAPI on Railway, rate-limited with slowapi
Frontend	Next.js + MapLibre GL	React + Vite (multi-page build) + Leaflet
Street View	Google Dynamic Street View	Not built
Auth	Google sign-in	Google sign-in, ID token verified server-side, own session cookie
Hosting	–	Railway (frontend and backend), Cloudflare DNS and email routing

Three changes are worth explaining.

The geocoder became a file. The plan was to turn “Walkley and Cedarwood” into coordinates with Nominatim. Nominatim does not understand intersections. It succeeded twice only because OpenStreetMap happened to contain named junction points, then failed on a junction that plainly exists. The replacement uses a fact already sitting in the map data: an intersection is a node shared by two or more named roads. One pass over a 47 MB Ottawa extract gave every junction, every street name and the link between them. Lookups became offline, instant and exact, and the street-name list came free with it. It also removed a dependency on free public services, which mattered: on the same afternoon Nominatim half-worked, Overpass timed out and a mirror returned HTTP 500.

No PostGIS. Railway’s standard Postgres image does not include PostGIS. The data is hundreds of points and lines, not millions, so a custom database image was not worth it. Coordinates are plain columns, route geometry is JSONB (it is only ever read whole, to draw), and PostGIS can be added later without a redesign.

React, Vite and Leaflet instead of Next.js and MapLibre. The site has a handful of real pages (map, discussion, about, legal) and no need for server rendering. Vite’s multi-page build serves each as its own entry point, and Leaflet with OpenStreetMap tiles needs no API key and has no metering.

2.2 Architecture

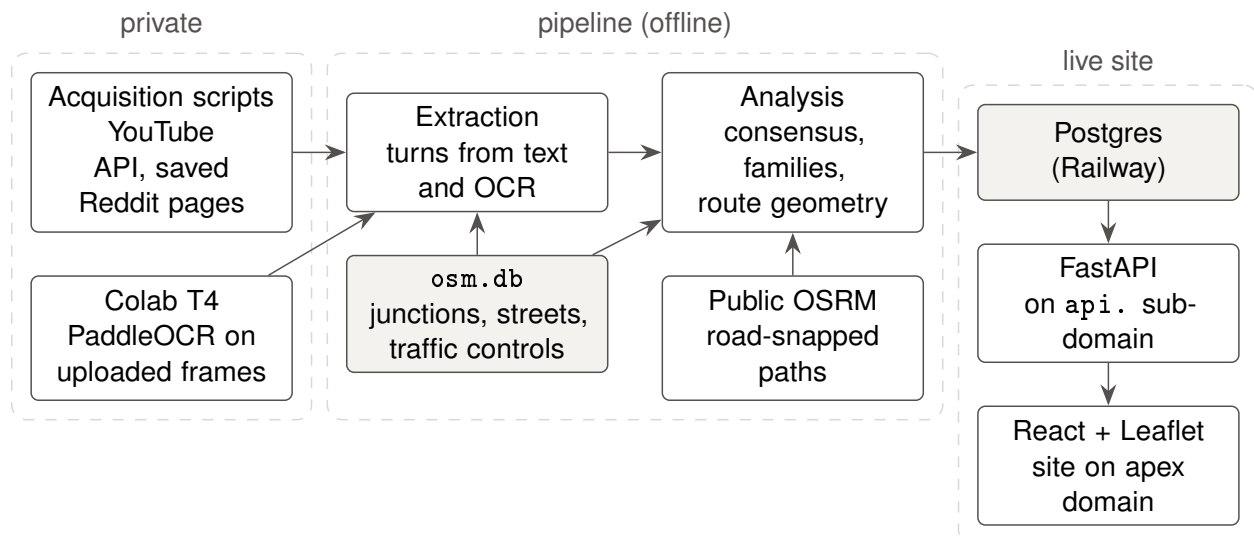


Figure 2: How data moves. Collection is private, because it holds API credentials and third-party material. The pipeline runs offline and writes to Postgres through an idempotent migration script. The live site only reads from Postgres, except for community posts and submissions.

The intermediate form is the contract between the halves. Every source, whether a Reddit comment or a dashcam video, becomes the same thing: an ordered list of (direction, street) turns, tagged with a pseudonymous author, a source type and a date. Everything downstream is source-agnostic. Reddit usernames are replaced at ingest by a salted hash, so no username is ever stored, yet two posts by the same person can still be recognised as one source.

3 Four weeks, in phases

4 The struggles

This is the longest section because it is where the project actually happened. The problems are grouped by kind rather than date. Most follow the same shape: something produced a confident, plausible result that was wrong, and nothing in the system noticed.

Table 2: How the project unfolded. Dates are 2026.

Dates	What happened
28–29 Aug	Planning docs, repo, API keys, domain. First collector run. Reddit access fails three ways.
30 Aug	112 Walkley videos collected. Source viability confirmed: roughly 36 real route recordings from about 20 channels. Local intersection index replaces geocoding.
31 Aug	First map geometry. A route that passed every check was missing a street (Section 4.3).
1–3 Sep	OCR made to work on dashcam footage. First route confirmed by two independent sources in different media. G videos processed on Colab.
4 Sep	Validation against baselines, shared street-name module, drive-past problem diagnosed, first drawable consensus route. Walkley done as a data exercise.
7–8 Sep	Canotek. Four bugs that could never have shown up at Walkley.
8–11 Sep	Smiths Falls. The map data turns out not to cover it; extract rebuilt. Both Reddit backlogs read in full. Winchester collected and abandoned.
12 Sep	Codebase cleanup. Postgres provisioned, schema designed, data migrated, FastAPI backend deployed.
13–15 Sep	Frontend. Making dots into routes, the predicted-bridge decision, turn-by-turn directions, stop signs and speed limits.
18–19 Sep	Hand-transcribed routes from real videos go live and replace the automated routes. Junction and wording fixes.
19–21 Sep	Feature backlog: badges, difficulty, reports, compare, French, GPX, submissions, forum, discussion, notifications, GPS drive-along. Three design passes. Security audit. Full frontend and backend deployment on custom domains.
25 Sep	Full redesign from a written spec, route playback, public read access, mobile practice drive tested on an iPhone, turn pins, domain outage, performance, production sweep.

4.1 Getting the data at all

Reddit, three ways. The first plan used Reddit’s public JSON search endpoints, on the reasoning that a one-off count did not need an API app. Every request returned HTTP 403: Reddit now blocks unauthenticated JSON access from most addresses.

What broke. Creating an official API app then failed silently. The form returned HTTP 200 with "success": true, no app appeared, and the console was clean. Three guesses went into form validation, ad blockers and the old Reddit interface.

What fixed it. Nothing, in the end. Reading the actual response body in DevTools showed the real message: app creation now needs approval under Reddit’s developer policy. A data-access request was filed and rejected overnight with a boilerplate reason. Reddit threads were collected by hand and parsed offline, which turned out to be better anyway: the saved pages held full comment text that an earlier CSV export had truncated.

The lesson was simple and kept recurring. A 200 does not mean it worked, and a shortcut that “doesn’t need approval” should be tested before anything is built on it.

The transcript library changed under the code. All 112 Walkley videos came back with “no transcript”. That looked like a finding about YouTube.

What broke. The youtube-transcript-api library had changed its interface in version 1.0, a static method became an instance method, and the script’s except block returned the string “none” without the error.

What fixed it. Detect whichever interface is present, and print failure reasons grouped by type.

The real picture: many videos have captions turned off, YouTube starts rate-limiting after repeated fetches, and even where transcripts exist, drivers say “turn left here”, not the street name. Transcripts were dropped as a route source.

Colab could only download 144p. OCR needs 1080p video. On Colab, every download capped at 144p, whatever cookies or player settings were used.

What broke. YouTube flags datacenter IP ranges and serves them degraded formats. The same cause was behind the transcript rate limits.

What fixed it. Split the job. Download and extract frames at home, where a residential connection gets full resolution. Upload the frame images to Drive, and run only the OCR on Colab’s GPU.

The Colab environment. Installing the GPU build of PaddlePaddle broke Colab’s preinstalled PyTorch with an NCCL symbol error. A fresh session silently reinstalled PyTorch and brought the error back. A leftover import in the setup cell initialised the OCR library twice. The free daily GPU allowance ran out mid-batch, and the retry session came back CPU-only. The fixes were unglamorous: uninstall PyTorch in the first cell every time, keep settings in the cell that uses them, save results to Drive after every video so a disconnect costs one video, and print progress every 200 frames instead of sitting silent for 25 minutes. For scale, one video processed on the laptop’s CPU took about 20 seconds per frame, roughly a hundred times slower than the T4, against an earlier guess of three times slower.

A video that narrated itself. Most dashcam videos never say a street name out loud. One Smiths Falls G2 video did something better: it burned an instruction for every turn into the picture (Figure 3). OCR read those captions line by line, and they became a complete trace that agreed turn for turn with an independent Reddit account written by someone else. It still needed one fix. Normalising “St.” replaced the full stop with a space, which left a double space and broke exact matching, so “St. Lawrence St” was never found.



Figure 3: Two frames from a Smiths Falls G2 recording, with the creator’s watermark cropped out. The burned-in captions (“Turn left on to Percy St from the stop sign”, “Three-point turn on Lavinia St”) were read by OCR and became the centre’s first complete video trace.

4.2 Silent failures

Several times the pipeline produced a clean, believable result that was completely wrong. These were the most expensive problems, because each one looked like a finding.

“Dashcam signs aren’t readable”. The first full OCR run reported 1,095 frames processed, zero with any text. That would have killed the main extraction path.

What broke. Three faults were stacked. PaddlePaddle’s oneDNN backend threw on every frame, and a bare `except:` `continue` discarded all 1,095 exceptions. Downscaling to 1280 px destroyed the small text on residential street signs. And cropping to the top of the frame removed signs that sit lower than assumed.

What fixed it. Disable the failing backend, count and print errors, and run at full 1080p with no crop. The same video went from 2 street hits to 114. The tell had been available from the start: dashcam footage always contains *some* text, so “zero frames with any text” was physically impossible.

Everything that silently wrote nothing. A consensus script had a `-report` flag meaning “print the table, write nothing”. It was passed alongside `-out` in every regeneration for two centres, so their output files sat five days stale. It surfaced only when the database migration showed zero rows. A snapping script defaulted its output path to Walkley’s file, so running it for any other centre would have overwritten Walkley. A Colab cell skipped unzipping frames whenever the folder existed, and an earlier failed run had created that folder empty. Each fix was small: make the output path required, rename or remove the ambiguous flag, and rebuild the folder every time.

Stale copies overwriting working files. Twice a whole script was rewritten from an older copy instead of the current file. The second time, 126 lines vanished, taking the centre lookup, the crescent fix and the carriageway fix with them, and reintroducing a bug that had taken three rounds to find. Git recovered the committed version, but everything applied since the last commit was gone and had to be rediscovered one failure at a time. The rule afterwards: never rewrite a file from memory, and read the current file first.

Nine screenshots of the desktop. A block of Python was pasted into the terminal instead of the editor. Bash ran `import argparse`, `import json` and the rest as ImageMagick’s `import` command, a screen-capture tool, and left nine 12 MB PostScript screenshots named after Python modules in the repo. Git’s status panel caught them before a commit.

4.3 Geometry: when a line on a map lies

The crescent that wasn’t driven. The first route drawn on a map was missing Fairlea Crescent. Every automated check said it was perfect: 10 of 10 junctions resolved, status “full”, plausible length, no warnings. It was caught by looking at the map, by someone who knows the neighbourhood. Fixing it took five rounds, each revealing the next problem:

1. Walkley Road is a divided road, so a rule picked the junction nearest the previous point. For a street you enter and leave, the previous point *is* a candidate, at distance zero, so both ends of the crescent collapsed into one.
2. A midpoint waypoint was added to force the route through the crescent, but its trigger fired on every pair of turns.
3. With the trigger corrected, the wrong midpoint was chosen, and the router drove in and turned around.
4. With the right midpoint, a routing flag set carelessly in the first draft still allowed a U-turn at every waypoint. A road test never asks for a U-turn.

5. With the flag removed, the route finally drove a proper loop.

The same session found that the test centre itself had never been modelled. Every route starts and ends there, but it is not a street, so it was being filtered out. It exists in OpenStreetMap as a `driver_testing` amenity, and now resolves like any other point.

Two routes welded into one. The router will happily connect any list of points. One Reddit account described two separate G routes with gaps between them, and the result was a smooth, entirely plausible 15.1 km route that returned to the test centre twice midway. Routes are now drawn only when every consecutive pair of streets resolves.

Invisible ramps. Airport Parkway, a multi-kilometre road, had exactly two junctions in the index. Highways connect through ramps, and OpenStreetMap ramps usually have no name, so a rule built to count only named roads threw every ramp connection away. Six people describing “Walkley to Airport Parkway” were being rejected as impossible. Grouping unnamed ramps into connected pieces fixed it.

Canotek: four bugs Walkley could never show. A second centre broke assumptions the first had hidden.

- **Accents.** OpenStreetMap stores *Montréal Road*, while everyone types “Montreal”. The backbone of the route was unreachable while four people described it correctly.
- **“East” is not a stopword** when the street is East Acres Road.
- **A one-row fallback.** The centres table had one row, and the lookup fell back to row 0. Every Canotek route resolved to Walkley, 12 km away, producing a 28.64 km route and a 0.00 km one.
- **Crescent rules on highways.** The midpoint rule, tuned on a 200 m crescent, sent the route 2.8 to 4.9 km off course on Blair Road and Airport Parkway. It is now capped.

The map didn’t reach the rural centres. The plan asserted that all four centres sat inside one Ottawa extract. A direct check found zero junctions near Smiths Falls. Worse, the pipeline would not have errored: every trace would have failed quietly, which looks exactly like the “sparse rural data” result the centre was meant to test. The extract was rebuilt from a Geofabrik Ontario download clipped to all four centres. That exposed another bug: centre IDs were derived from the OpenStreetMap name tag, and two centres were both named “DriveTest”. Centres are now matched to known coordinates, and Walkley’s junctions were confirmed identical in both databases before the swap.

Same name, different town. A province-wide street list means “Main”, “Queen” or “Jasper” matches roads all over Ontario. One route blew out to 170 km before the junction picker was limited to the nearest match within 15 km. A promising “bridge” between two Smiths Falls routes turned out to be a Main Street elsewhere in the region, and “Colonel By” on a map was Ottawa’s, not a Smiths Falls street.

4.4 The same bug, five times

Street-name normalisation lived in five scripts under five names (`clean_street`, `normalise`, `base_name`, `base`, `base`). The same bug appeared in each: stripping a trailing “Parkway” turned Airport Parkway into “airport”, a different road inside the airport. The fourth time, the guard had been written but never called. The fifth time, a classifier treated “airport” and “airport parkway” as two competing streets at one junction. The real fix was one shared module that every script imports.

The pattern repeated with a tuned parameter. After the age-decay half-life was changed from 3 to 5 years in two scripts, the script that builds the published map still used 3. It changed no published street, but the numbers being trusted and the map being drawn came from different settings. A test-class voting function had been copied into two places, where “ambiguous” traces were outvoting confirmed ones. Both were consolidated the same way.

4.5 Metadata that lied

- **The centre column** stamped the search term onto every result. All 94 “Walkley” videos said Walkley, including ones titled Canotek and Smiths Falls.
- **SEO hashtag blocks.** Driving schools append dozens of town hashtags to every description, so videos about Brockville and Kingston were attributed to Smiths Falls. Hashtags are now stripped before matching, and descriptions are saved so every attribution can be audited.
- **Winchester, England.** Of 16 “Winchester” videos processed through the full download and OCR pipeline, all 16 were from Winchester, Hampshire. The OCR text named UK roads and driving schools, and one frame showed a GPS readout at N51 / W1. A title filter had caught 15 UK videos earlier, but the rest gave themselves away only in the footage. Winchester, Ontario’s own Reddit threads named no streets at all. The centre was skipped. The better fix for next time is to put “Ontario” in the search query itself.

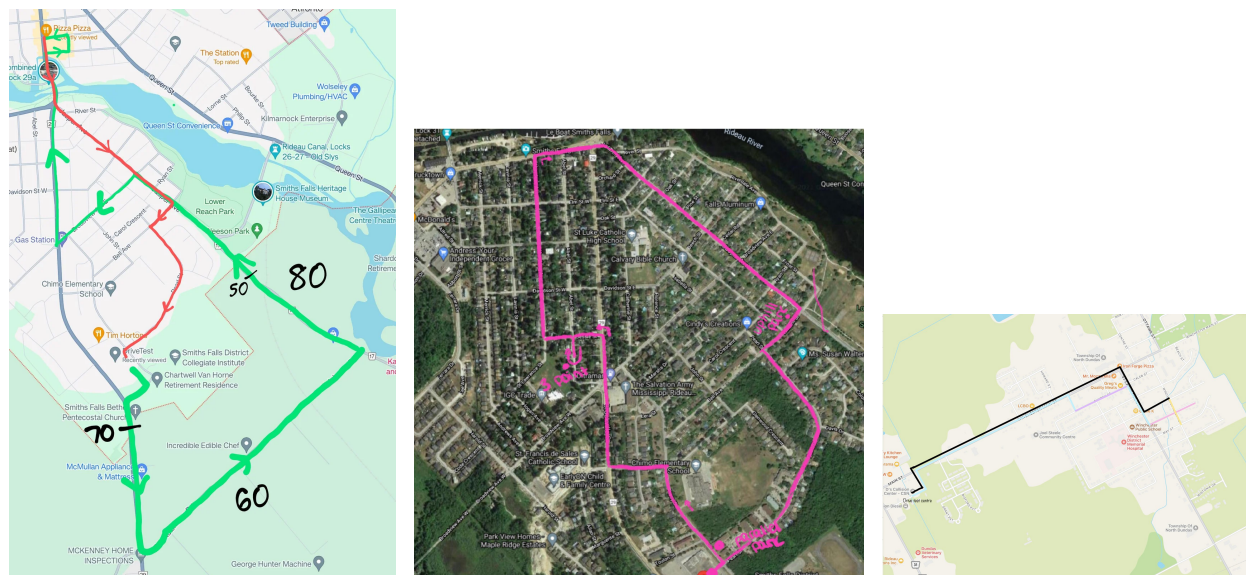
4.6 Evidence that came as pictures

Some of the most useful evidence came as maps people had drawn on and shared publicly (Figure 4). These were read by hand, never by the pipeline, and every street pair taken from one was checked against the junction index before anything was built from it.

A Smiths Falls G map traced a loop out past the Rideau Canal locks, marked with speed zones of 50, 80, 60 and 70. At first it seemed to contradict accounts of “two seconds at 80 on Jasper”. It didn’t. Those accounts described the short 80 zone inside the same loop, which a shared Google Maps route later confirmed. An annotated satellite map of the G2 loop marked where the three-point turn and the parallel park happen. A Winchester image traced several routes in different colours with no legend. Nothing said which line was which, so it was set aside rather than guessed at.

4.7 The street the route drives past

The best-supported street in the whole Walkley corpus was Colliston: seven independent sources, fifteen video sightings. The route never turns onto it. It passes straight through a signalised intersection whose overhead signs name both Colliston and Albion. The sign is real, the junction is real and the read is exact, so every video filter passes it. The only thing that separates it is text: none of the people who wrote out that route mentioned Colliston. *A camera records what it passes; a person records what they did.* It was fixed with a manual correction, which is exactly what the corrections file exists for.



(a) Smiths Falls G loop with speed zones. (b) Smiths Falls G2 loop, manoeuvres marked. (c) Winchester, several unlabelled lines.

Figure 4: Route maps shared publicly by test-takers. They were used as leads and checked against the road graph, never ingested directly.

4.8 From dots to a route people can read

The first working app, on 13 September, was bare HTML: a list of centres with their counts, and a map of blue junction dots (Figure 5). At Canotek a few confirmed runs were drawn, surrounded by dots that joined nothing. Some of those dots sat on the Orléans side of the centre and came from traces too few to form a route family, which is the minimum the pipeline required before trusting a cluster as a route.

Straight lines through buildings. Early maps showed confirmed junctions as disconnected dots. The first attempt to connect them drew straight lines between points that share a street name, and the lines cut across blocks and through buildings. It was reverted, visibly, in git history.

The predicted-bridge decision. Connecting confirmed points along real roads makes a route readable, and also draws road no source described. The worry was concrete: a predicted turn that looks confirmed is more dangerous than one that is obviously wrong, because someone might drive it on a real test. The rules that made it acceptable were these. Bridge only between points already in the same verified route family. Use the fewest bridges possible, with a distance cap. Flag every bridge in the data. Draw bridges in a style that never looks confirmed.

The first pass still left dots unconnected. The cause predated it: a run of a single segment can never become a line, so fully confirmed one-segment branches had always been invisible. Another gap was real evidence being thrown away: sources saying "... then back into the test centre", which the geometry step skipped because the centre is not a street.

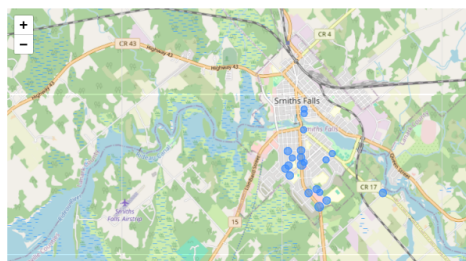
"G and G2 look the same". Clustering grouped routes by shared streets and ignored test class, so a G route and a G2 route sharing roads near the centre merged, and both showed under both filters.

OntarioDriveTestMap

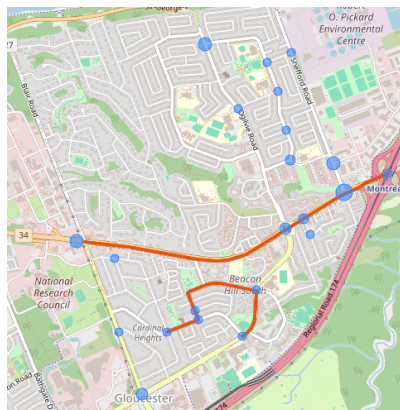
Signed in as [email redacted]

Centres

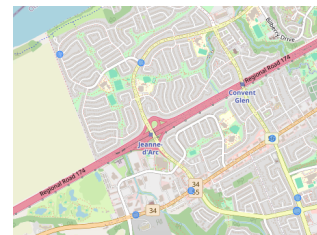
Ottawa Canotek	— 23 traces, 29 segments, 2 route lines
Smiths Falls	— 7 traces, 20 segments, 0 route lines
Ottawa Walkley	— 26 traces, 52 segments, 8 route lines
Winchester	— 0 traces, 0 segments, 0 route lines



(a) The unstyled first app (email redacted).

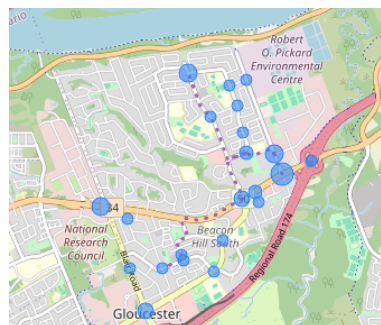


(b) Canotek: short runs, many loose dots.

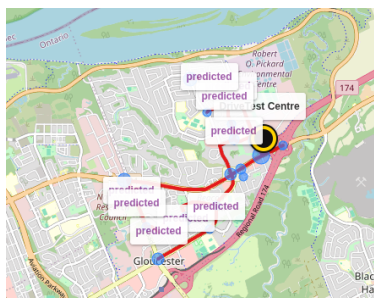


(c) Orléans dots, never a family.

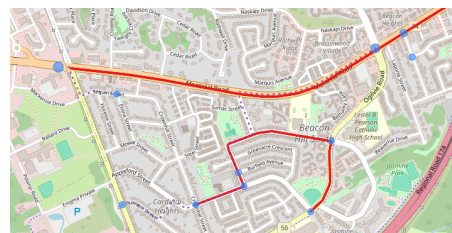
Figure 5: 13 September. Confirmed junctions existed, but most of them were not yet connected into anything a reader could follow.



(a) Predicted bridges as purple dots.



(b) Every predicted piece labelled.



(c) Confirmed Canotek geometry in red.

Figure 6: Iterations on showing inference. Predicted connections always had their own style. Confirmed data dropped per-class colours for a single red, and the final design replaced all of it with one notation: Confirmed, Inferred and Unrouted gap.

Clustering now runs within each class. On the frontend, react-leaflet's GeoJSON component reads its data only once, at mount. Switching routes updated the sidebar while the drawn line stayed frozen. That was caught only because the report of “every route looks identical” was not accepted with a “try a hard refresh”. A key per route fixed it.

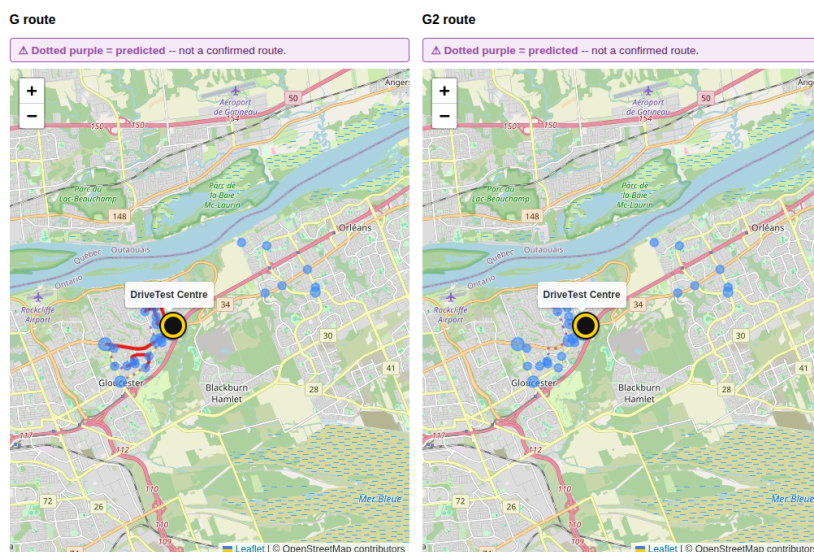


Figure 7: A rejected layout: G and G2 side by side. The junction dots carry no class, so both panels showed the same dots and read as a duplicate. It went back to one map with a G/G2 switch.

Directions that read like directions. The first directions table copied a competitor's layout (Figure 1) but was built from the router's own steps, and it read badly (Figure 8). The router treats every waypoint as a destination, so “Arrive at destination” appeared every few rows, and every predicted row carried an inline tag. Now only the true final arrival survives, intermediate arrivals become “Continue onto”, and predicted rows are shaded instead of labelled. Stop signs, traffic lights and speed limits come from real OpenStreetMap tags: 115,196 stop signs and 23,213 signals across the Ontario extract. The column is left blank where nothing is tagged, because blank means “untagged”, not “none”.

Why the routes on the map are hand-transcribed. Automatic clustering cannot tell three short routes from one long route seen in pieces. Fixing route counts by hand and letting a travelling-salesman tour order the junctions gave connected routes, and a 36 km Walkley G loop, longer than any real test. The answer was to transcribe real test videos street by street, by hand. That overturned the assumed counts: Walkley G has three routes, and Canotek has two G and three G2. Resolving the transcripts still surfaced traps. Fairlea Crescent meets Heatherington twice, 200 m apart, and the transcript meant the second. A “Regional Rd” was Jasper Road becoming Jasper Avenue. The router's generated instructions silently left out short streets that had been forced into the path, such as a parking-manoeuvre cul-de-sac. The written directions now use clean wording based on the transcript, with real per-leg numbers aligned to each line. Speed limits are looked up by location, not by street name, because Walkley Road is 50 near the centre and 80 toward the airport.

Distances and durations are calculated by routing software between the collected data points, not measured from a live drive.

#	Instruction	Distance	Duration
1	Head onto Ogilvie Road	413 m	1 min
2	Continue onto Elmridge Drive	810 m	2 min
3	Continue onto Appleford Street	275 m	1 min
4	Arrive at destination	0 m	0 min
5	Head onto Montréal Road	2.65 km	4 min
6	Arrive at destination	0 m	0 min
7	Head onto Elmridge Drive (predicted)	79 m	1 min
8	Arrive at destination (predicted)	0 m	0 min
9	Head onto Blair Road (predicted)	200 m	1 min
10	Turn left onto Séguin Street (predicted)	197 m	1 min
11	Arrive at destination (predicted)	0 m	0 min
12	Head onto Appleford Street (predicted)	140 m	1 min
13	Turn left onto Corliss Street (predicted)	131 m	1 min
14	Turn right onto Dunham Street (predicted)	225 m	1 min
15	Arrive at destination (predicted)	0 m	0 min
16	Head onto Séguin Street (predicted)	197 m	1 min
17	Turn left onto Blair Road (predicted)	619 m	1 min
18	Arrive at destination (predicted)	0 m	0 min
19	Head onto Montréal Road	340 m	1 min
20	Turn slightly right onto the road	102 m	1 min
21	Continue straight onto Shefford Road	137 m	1 min
22	Turn right onto Canotek Road	109 m	1 min
23	Turn left onto the road	40 m	1 min
24	Turn right onto the road	8 m	1 min
25	Arrive at destination	0 m	0 min
26	Head onto Kender Avenue	353 m	1 min
27	Arrive at destination	0 m	0 min
28	Head onto Eastvale Drive	832 m	2 min
29	Arrive at destination	0 m	0 min
30	Head onto Casey Avenue (predicted)	212 m	1 min

Figure 8: The first directions table: repeated “Arrive at destination” rows and inline “(predicted)” tags.

4.9 Building the community side

A submission form rebuilt four times. The route-submission form went through four versions, each prompted by a specific complaint. The first was an inline form that checked the whole street list against the road graph only on submit, then worked out which pair had failed by parsing an error string, which would break on any street name containing a quote. It became a header button that works for any Ontario centre, including ones the site has no data for yet, because the street index covers the whole province. Then it became a four-step wizard (centre, class, streets, review) that checks the first street, and then each new street against the one before it, the moment it is typed. Finally each turn gained a junction type (light, stop sign, roundabout and so on) so the review screen reads like the site’s own directions. Submissions live in their own tables, deliberately apart from the pipeline’s traces table, because the migration script deletes and rewrites that table on every rebuild and would have silently wiped them.

Moderation with no moderators. The forum had a hard constraint: no AI moderation service and no human moderators, not even volunteers. That led to the model that large community sites actually use. Posts hide automatically at a score of -5 or after three reports, computed when the page is read so the rule can never drift from the votes. A local word list, and daily limits of 10 posts and 30 comments per account, cover the rest. An admin role with delete power came later, which reversed the original constraint. That reversal was written down in the code rather than slipped in quietly. Images are stored in Postgres itself, with a type whitelist and a 5 MB cap, because the project has no object storage and adding it was not worth one feature. Notifications for mentions and replies are in-app only, since the site sends no email.

A privacy policy that described a different company. A policy generator produced a draft that claimed the site used marketing partners, data brokers, advertising cookies and social-network friend lists. None of that is true of this site. One clause also contained an explanatory aside that had leaked into the legal text. The policy was rewritten against an audit of what the app actually collects. The lesson: generators default to the most common kind of website, an ad-supported one, and this site is not that.

4.10 Smaller bugs that were still real

- **Inflated driving times.** A 3.04 km route showed about 14 minutes. One leg's duration had been copied onto every direction line covering that leg, and the summary added them all up. After de-duplicating it shows about 6 minutes.
- **Half the confirmed routes hidden.** The map filter matched a route's class to the G or G2 button. Routes whose class was honestly unknown matched neither, so 4 of Walkley's 8 confirmed routes never drew.
- **One error broke every request after it.** The database driver left a transaction open when a query failed, and the broken connection went back into the pool. A single glitch then broke every later request until a restart.
- **Speed limits one exception away from vanishing.** One try block wrapped both a new location-aware lookup and its fallback. If the new table was missing, every speed limit on every route would have gone blank instead of falling back.
- **"St." as a first word.** Expanding "St" to "Street" turned "St. Lawrence St" into "street lawrence", so matching keyed on the word "street". At the start of a name it now means Saint.
- **Twenty-six routes instead of thirteen.** Re-running the route import deleted only rows from the previous source, so a second run duplicated everything. It was caught by a row count immediately after applying.
- **A URL that matched the wrong route.** `/centres/compare` was registered after `/centres/{id}`, so FastAPI treated "compare" as a centre ID. It was caught by dumping the registered routes before shipping.
- **Signed in but not recognised.** The Google login response returned only name and email, so an admin account looked like a new user until a page reload.
- **White text on white.** Turning on dark mode automatically made the directions table unreadable, because its row backgrounds were hard-coded light. Dark mode was put behind an explicit switch until every component used the colour tokens.

4.11 Shipping it

Three deploy crashes in a row. The first backend deploy crashed three ways in sequence. `psycpg2-binary` had no wheel for Railway's default Python 3.13 and failed to compile, so Python was pinned to 3.12. `requests`, needed by Google's auth library, had only ever been installed as a side effect on the laptop. Then the service had zero environment variables and crash-looped on a missing database URL.

The backend imported files that weren't there. Railway builds only the `backend/` folder, but the backend imported pipeline modules from the repo root and read an 18.6 MB `osm.db` that was gitignored and never committed. The modules and a copy of the database were vendored into `backend/`, and the fix was verified by running the folder alone, the way Railway would. The frontend had its own trap: a `vite` version constraint that the lockfile had been silently masking broke a clean install.

Logged out on every click. With the site on its real domain and the API on Railway's domain, the session cookie's `SameSite=Lax` policy blocked it on every cross-site request, and the app looked logged out after every click. It had worked locally only because two localhost ports count as the same site. The fix kept the cookie policy strict: the API moved to `api.ontariodrivetestmap.fyi`, the same site as the frontend. Google sign-in then failed with `origin_mismatch` until the new domain was added in Google Cloud Console.

Static pages that all showed the home page. The frontend was served with `serve -s`, which rewrites every request to `index.html`. The About, Privacy, Terms and Cookies pages all quietly showed the home page. Removing `-s` fixed that, but its clean-URL redirects then dropped query strings, which broke "discuss this route" links. The first fix for that exposed a directory listing at `/.`. The final version uses explicit rewrites with directory listing off. Railway also reused a cached build after environment-variable changes, so new values did not take effect without a real source change.

A security audit that found something real. An adversarial audit found one high-severity issue. The public analytics endpoint accepted a page path that was later placed unescaped into an admin digest email, a stored HTML-injection path from an anonymous visitor into an admin inbox. The template is now escaped and the path restricted at the source. The same pass added rate limiting across about 45 endpoints, length caps on every user text field, security headers, and session revocation on account deletion.

Slow, then down. API calls took 0.6–1.3 s even for tiny responses, because the backend reached Postgres through Railway's public proxy instead of the private network. Switching to the private address, a thread-safe connection pool, GZip (93 KB of map JSON down to 14 KB), short server-side caching and a leaner first page fixed it. Separately, the site went down entirely when its custom domain was detached from the frontend service and the DNS record deleted. The fix was re-attaching the domains in Railway and restoring the DNS records.

4.12 Scope decisions

- **Appointment-opening alerts** were the most-wanted idea and the least buildable. DriveTest's booking system requires a driver's licence number to see anything, allows 10 login attempts a day, and takes payment at booking. Automating logins on someone's behalf was ruled out. It shipped as an in-app reminder to go and check.
- **Rate My Examiner** was dropped. Naming real examiners carries defamation risk that a free side project should not take on.
- **Reading is public.** The site originally required sign-in to see a route. That was reversed: guests read everything, and sign-in is asked for only when posting.

- **No ads.** AdSense would have offset about \$10 a month at the cost of consent banners and the site's honesty positioning.
- **Design, three times over.** Two colour directions were rejected, and the redesign that followed changed layout, not just paint: a persistent map beside a scrolling route panel, modelled on how real route planners work. A later full redesign from a written spec settled the one-notation confidence system, and a serif headline font was cut on sight.

4.13 Process

Twice the work stalled in ways that had nothing to do with code. Three sessions in a row ended with a longer audit list than they began with: 15 items, then 9, then 7. None of them converged, because each list came from re-auditing rather than from tracking what was actually open. It stopped when a fixed completion bar was applied and the centres were declared done against it. And for long stretches, working changes sat uncommitted, so a bad overwrite could only be recovered to a stale version. Committing more often would have made every recovery cleaner.

5 What shipped

As of 25 September 2026 the site is live at ontariodrivetestmap.fyi. Its API runs on the `api` subdomain, and both are on Railway with valid certificates.

- **13 routes across three centres:** Walkley (3 G, 3 G2), Canotek (2 G, 3 G2) and Smiths Falls (1 G, 1 G2). Each was transcribed street by street from a real test video, resolved against the local junction index and routed on real roads. Of the transcribed turns, 92 of 126 are pinned to a real junction on the map, in driving order on every route. The rest stay text-only rather than being placed somewhere approximate.
- **The evidence layer.** Crowd-sourced junctions sit under each route, off by default, with a source drawer that counts videos and community reports. The confidence notation reads the same on the map, the legend, the turn list and the centre index.
- **Community features** are live for signed-in users. Submissions are validated against the road graph as they are typed, and a promotion script can turn independently corroborated submissions into routes, held to the same bar as everything else.
- **Mobile practice drive**, tested on an iPhone 11 over a secure tunnel, with a simulated-GPS mode for testing without driving.
- **Production basics:** per-page SEO, a sitemap generated from real data, a privacy policy checked line by line against what the app actually does, rate limiting, security headers, account deletion by anonymisation (so other people's threads survive), and a `www`-to-apex redirect.

What's next. Three directions follow naturally from where the project stands. The first is spoken, turn-by-turn guidance in the practice drive. It was held back deliberately until every turn has a verified position on the map, and the turn anchors built last are the first step toward it. The second is coverage: more centres, starting with a second attempt at Winchester using searches scoped to Ontario. The third is official data. A freedom-of-information request to the Ministry of Transportation for current pass rates and failure reasons is pending. Nothing on the map comes from the Ministry today.

6 What I would keep, and what I would do differently

Validate the instrument before the result. The costliest bugs never crashed. They returned zero, “none”, or a route that looked right. The tell was nearly always a number that was too clean, like zero frames with text or zero transcripts out of 112. Errors should be counted and printed, never swallowed into a status string, and an impossible result is a bug report, not a finding.

Look at the map. Fairlea Crescent passed every check. Colliston had the most support in the corpus. A route welded from two others looked plausible. Each was caught by a person looking at a rendered map who knew the area. Automated checks measure what they were written to measure, and a map shows everything at once. The reverse also held: two “this looks wrong” moments turned out to be label placement at a particular zoom, and only checking the underlying coordinates settled them.

A new centre is the best test. Canotek found four bugs in one day that Walkley could never have exposed: accents, direction words, a one-row fallback and a rule tuned on a 200 m crescent. Smiths Falls found that the map did not cover it. Each new place tests the assumptions the last one hid.

The second copy is the signal. The normalisation bug was fixable the first time it appeared in two files. It came back three more times because the second occurrence was read as coincidence. The same was true of a tuned parameter and a voting function. When logic exists in two places, move it into one.

Derive attributes from content. Every field a collector assigned without checking turned out wrong somewhere: centre, test class, even country. Street signs read from a frame are better evidence of where a video was filmed than its title is.

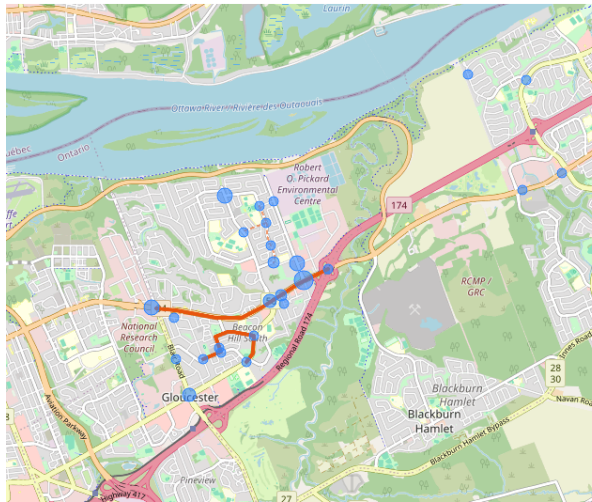
Make inference visible. Drawing road that no source described is sometimes the right product decision. It is only acceptable when it is flagged in the data and drawn so it can never be mistaken for confirmed evidence. A predicted turn that looks confirmed could send someone the wrong way on their actual test.

Stop auditing, start finishing. Three sessions produced longer to-do lists than they started with. A fixed, written bar for “done” ended that in one step. Beyond that bar, the honest way to find out what is still missing is to build on the data, not to keep inspecting it.

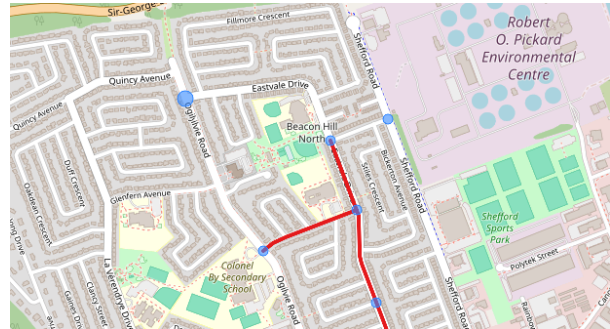
Local knowledge is the scarcest input. Knowing that the car goes straight at Colliston, or that a transcript meant the second Fairlea junction, fixed things no pipeline could. It is also the least checkable input, so it needs more care, not less. When the scope of a local correction is unclear, record the narrow version: one correction recorded too broadly wrongly marked a real route as disputed.

A More screenshots

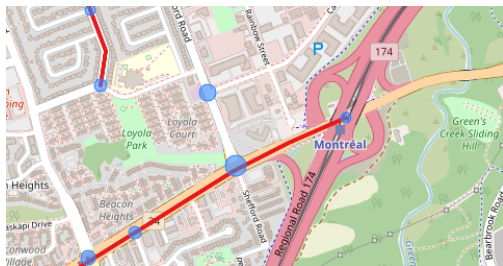
Screenshots taken on 13 September 2026, the day the map first started to look like routes. They sit alongside the figures in Section 4.



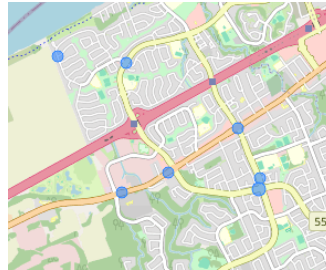
(a) Canotek at city scale: confirmed runs near the centre, loose dots beyond.



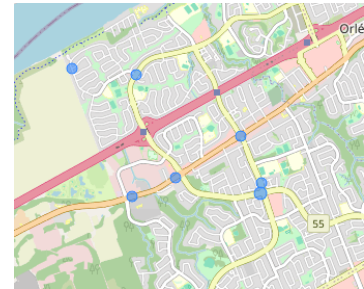
(b) Ogilvie Road and Eastvale Drive, confirmed geometry.



(c) Montréal Road at the Highway 174 interchange.

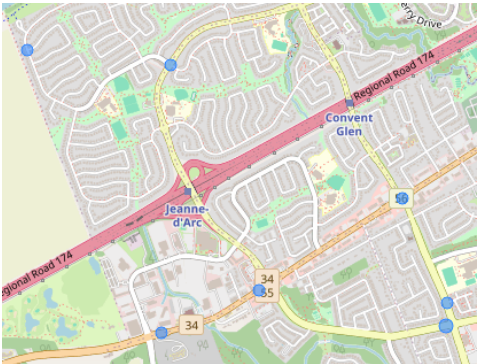


(d) Orléans dots.



(e) Orléans dots, later render.

Figure 9: Canotek on 13 September.



(a) The Orléans cluster, still unbridged after the bridge pass.

#	Instruction	Distance	Duration
1	Head west on Canotek Rd	0.10 km	1 min
2	Turn right onto Shefford Rd	0.71 km	1 min
3	Head north on Shefford Rd toward Casey Ave	0.28 km	1 min
4	Turn left onto Casey Ave	0.21 km	1 min
5	Turn right onto Eastvale Dr	0.63 km	1 min
6	Turn left onto Ogilvie Rd	0.27 km	1 min
7	Head south on Ogilvie Rd toward Glenfern Ave	0.05 km	1 min
8	Turn right onto Glenfern Ave	0.39 km	1 min
9	Turn left onto La Vérendrye Dr	0.17 km	1 min
10	Turn right onto Meldrum Ave	0.10 km	1 min
11	Head west on Meldrum Ave toward Clancy St	0.18 km	1 min
12	Turn left onto DeLong Dr	0.39 km	1 min
13	Turn left onto Rothwell Dr	0.45 km	1 min
14	Turn right to stay on Rothwell Dr	0.28 km	1 min
15	Turn right to stay on Rothwell Dr	0.14 km	1 min
16	Turn left onto Cedar Rd	0.21 km	1 min
17	Turn left at the 1st cross street onto Beckenham Ln	0.12 km	1 min
18	Turn left onto Montréal Rd/Ottawa 34	0.56 km	1 min
19	Head east on Montréal Rd/Ottawa 34 toward Bethamy Ln/Foxborough Private	0.44 km	1 min
20	Head northeast on Montréal Rd/Ottawa 34	0.79 km	2 mins
21	Turn left onto Shefford Rd	0.23 km	1 min
22	Turn right onto Canotek Rd	0.14 km	1 min

(b) The competitor’s turn-by-turn table used as a layout reference.

Figure 10: The Orléans cluster, and the directions layout this site started from.